AX 전환
기술공유 플랫폼 로드맵

DAMI Lab
AX 전환 점검 세미나

2026.04.23

Contents

1. 비전과 문제
2. 시스템 4 Layer
3. 기술 아키텍처
4. 작동 예시 — 시나리오
5. 단계별 로드맵
6. 역할과 의미

1. 비전과 문제

한 줄 비전

■ 명제

- 개인이 만든 스킬이 **자동으로 포크·발전·공유**되며, 그 진화 과정이 홈페이지에 **족보로 남는 생태계**
- 「공용 라이브러리 하나 만들고 모두 쓰세요」 는 **개인화 욕구 무시**
- 「각자 알아서 하세요」 는 **공유 효과 0**

■ 해답의 구조

- **개인이 중심**, 단 포크·승격·알림으로 공유가 **자연스럽게** 일어나게
- 스킬은 개인적이면서 집단적인 두 얼굴 — 둘 다 잡는 설계가 핵심

개인적 속성

- 내 워크플로우 · 연구 분야 · 손버릇에 맞아야 쓸 수 있음
- 남의 것 그대로는 잘 안 맞음

집단적 속성

- 남의 것을 **기반으로** 하면 혼자 만드는 것보다 훨씬 빠름
- 진화 과정이 보여야 재사용 가능

핵심 설계 원칙: 포크는 자유, 승격은 합의. 둘을 구분하는 순간 개인화와 공유의 긴장이 풀린다.

설문이 드러낸 Pain Point

■ 현재 문제

- 지금은 「각자 만들고 슬랙에 올린다」 → **중복만 쌓이고 맥락이 사라짐**
- 다섯 가지 pain point 가 모두 **한 가지 구조 부재**에서 나옴

문제	설문에서 나온 표현
중복	스킬이 겹친다, 비슷한 걸 다 따로 만든다
발견 실패	슬랙/홈페이지에 올라오는 스킬을 어떻게 적용해야 할지 모름
이해 실패	정보가 너무 많다, 뭘 어떻게 쓸지 모름
동기화 실패	남의 것을 복사해서 쓰다 보면 원본 업데이트를 반영 못 함
부담 편중	일부 인원(특히 민석)에게 책임이 집중됨

근본 원인: 각자 자유롭게 만들되 **진화 과정이 자동 추적되는 시스템**이 없음. 이 덱의 §2 ~ §4 가 그 시스템의 설계.

2. 시스템 4 Layer

4 Layer 개관

■ 설계

- 네 개의 Layer 가 **아래에서 위로** 쌓이며 서로를 지탱
- Metadata 없이 Directory 없고, Directory 없이 Promotion 없고, 앞 세 층 없이 Visualization 없음

Layer 1
Metadata
frontmatter 강제
족보 · 상태 · 사용법

Layer 2
Directory
shared / personal / archive
Git 히스토리로 추적

Layer 3
Promotion
개인 → 공용 승격
워크샵 합의 기반

Layer 4
Visualization
홈페이지 족보 · 알림
외부 창구

핵심: Layer 1·2 는 **기술적 토대**, Layer 3 은 **사회적 프로세스**, Layer 4 는 **외부 창구**. 순서대로 구축.

Layer 1. Metadata — Frontmatter 강제

■ 원칙

- 모든 스킬 파일 맨 위에 필수 정보 — 족보를 만드는 열쇠는 `parent`
- `why` 와 `how-to-use` 는 **사람이 직접** 적어야 한다 (설문 요구사항)

```yaml
---
name: paper-summary-figures
version: 1.0.0
author: minseok
parent: yunji/paper-summary@1.2      # 포크 대상 / 없으면 null
status: personal                      # personal | candidate | shared
tags: [paper, summary, figure, research]
description: 윤지의 paper-summary 에 figure 자동 추출 기능 추가
why: 논문 읽을 때 figure 만 따로 보고 싶은 경우가 많음
how-to-use: |
  pdf 경로를 주면 summary.md + figures/ 폴더 자동 생성
---
```

parent 스킬의 족보를 만드는 열쇠 — 누구 걸 포크했는지 명시

status personal(개인) → candidate(승격 후보) → shared(팀 공식)

필수 **why** 와 **how-to-use** 는 사람이 직접 작성 — 자동 생성 금지

Layer 2. Directory Structure

■ 구조

- `shared/` — 팀 공식 (승격된 것만, 워크샵 승인 필요)
- `personal/<name>/` — 본인 완전 자유 구역
- Git 히스토리가 **모든 변경을 자동 추적**

트리

```
damilab-skills/
├── shared/
│   ├── paper-summary/
│   ├── daily-report/
│   └── proposal-bot/
├── personal/
│   ├── damilab/          # 교수님
│   ├── minseok/
│   ├── yunji/
│   ├── yunseok/
│   ├── seunghyun/
│   └── robot-team/
├── archive/              # 폐기·옛 버전
└── .damilab/
    ├── frontmatter-schema.yaml
    └── validator.py
```

규칙

- `shared/` 직접 쓰기 → **워크샵 승인**
- `personal/<name>/` → **완전 자유**
- `archive/` → 폐기·옛 버전 보관
- `.damilab/` → 스키마·검증 스크립트

경계 설계: 자유 구역(personal) 과 합의 구역(shared) 을 **물리적으로 분리**. 규칙 위반이 원천적으로 어렵게.

Layer 3. Promotion — 사회적 승격

■ 프로세스

- 개인 → 공용으로 올라가는 길은 **사회적 합의**. 자동 승격 없음
- 단순한 파일 이동(`git mv`)이라 기술적으로는 쉬움

① **만들기**
`personal/<name>/foo` 에 본인 스킬 작성

→

② **체류**
본인 몇 주 사용,
다른 사람 1~2명 포크

→

③ **제안**
매주 목요일 워크샵에서
공용 승격 제안

→

④ **이동**
합의되면 `shared/foo` 로
`git mv` , 포크한 사람들에게 알림

왜 자동 승격 안 하나: 「많이 쓰인다」 가 반드시 「공용 가치」 는 아님. 팀이 **어떤 범위까지 공식화할지** 판단해야 함.

Layer 4. Visualization — Homepage

■ 구성

- `share.damilab` 에서 메타데이터를 읽어 **자동 렌더링**
- 스킬 카탈로그·상세·족보·알림·코멘트의 다섯 축

카탈로그 전체 스킬 리스트, 태그 · 저자 · 상태 필터

상세 페이지 description · why · how-to-use · 사용법 · 부모 · 자식

족보 트리 `yunji/paper-summary → minseok/paper-summary-figures → damilab/paper-summary-v3` 나무 시각화

Update Alert 포크한 부모가 업데이트되면 자동 알림 — 머지는 본인 선택

토론 · 코멘트 각 스킬 페이지에 이슈 · 사용기 · 매주 목요일 워크샵 기록

핵심: Layer 1·2·3 의 산출물을 **한 곳에서 눈으로 볼 수 있게** 하는 것이 홈페이지의 역할. 글쓰기 도구가 아니라 **렌더러**.

3. 기술 아키텍처

Git-backed CMS

■ 흐름

- 저장은 **Git**, 글쓰기는 **홈페이지** — 두 층을 분리
- 연구원은 git 을 몰라도 됨, 뒤에서 자동으로 커밋·푸시

① **웹 에디터**
연구원이 홈페이지에서
노션 쓰듯 글 작성

→

② **Auto Commit**
백엔드가 자동으로
`git commit && push`

→

③ **Source of Truth**
git repo 가
모든 버전의 단일 진실

→

④ **Rebuild**
정적 사이트 재빌드
→ 홈페이지 렌더

CMS 도구 후보 Sveltia CMS (1순위) · TinaCMS · Decap CMS

왜 Git 인가: 모든 버전 히스토리가 자동 관리되고, Claude Code 가 로컬에서 그대로 사용 가능 (`git pull` 하면 끝). 별도 API 불필요.

메타 스킬 — 워크플로우 자체를 스킬화

■ 개념

- 자주 일어나는 동작(포크 · 커밋 · 승격)을 **그 자체가 스킬**이 되게
- 사용자는 Claude 에게 자연어로 말하면 메타 스킬이 규칙에 맞게 자동 처리

damilab-skill-fork 포크 자동화
- 복사 · frontmatter 채움 · 브랜치 · 커밋 · 푸시까지 한 번에

damilab-skill-promote 승격 자동화
- `git mv` · 버전 bump · changelog · 기존 버전 superseded 마킹

damilab-skill-commit 커밋 자동화
- 변경사항 요약 → 커밋 메시지 자동 생성

damilab-skill-search 검색
- 홈페이지 API 질의로 비슷한 스킬 · 부모 후보 탐색

포인트: 사용자는 「포크해줘」 · 「승격해줘」 · 「비슷한 스킬 있나?」 라는 자연어만 쓰면 됨. 규칙은 스킬 안에 박혀 있음.

4. 작동 예시
— 시나리오

예시 1·2. 포크와 업데이트 알림

■ 두 시나리오

- **포크**: 민석이 윤지의 스킬을 확장하고 싶을 때 — 메타 스킬이 전부 처리
- **업데이트 알림**: 부모 스킬이 진화하면 자식 스킬 소유자에게 알림 — **강제 아닌 선택**

① 포크 (민석 → 윤지)

민석 「윤지가 만든 paper-summary 를 포크해서 figure 기능 추가할게」

- fork 메타 스킬이 자동 실행
- `yunji/paper-summary@1.2` 식별
- 브랜치 · 복사 · frontmatter · 커밋 · 푸시
- **민석은 git 을 한 번도 안 침**

경험: 「포크해줘」 한 마디로 족보에 자식 노드가 추가됨.

② 업데이트 알림

상황 `shared/paper-summary` 가 1.0 → 2.0

- 홈페이지가 @1.0 을 parent 로 하는 personal 스킬 스캔
- Slack 봇이 각 소유자에게 알림
- 변경사항 요약 · 머지 명령 제시

원칙: 강제 업데이트 X. 본인이 결정, 머지도 Claude Code 가 diff 기반 도움.

예시 3·4. 승격과 신입 온보딩

■ 두 시나리오

- **승격**: 매주 목요일 워크샵에서 사회적 합의로 `shared/` 이동
- **신입 온보딩**: 시스템 자체가 **우리 연구실 매뉴얼**이 됨

③ 승격 (워크샵 합의)

민석 「paper-summary-figures 공용 올려도 될 듯. 3주간 15번 썼고 2명 포크함」

- 교수님 · 팀 논의 → 보강 포인트 합의
- promote 메타 스킬 실행
  - `git mv` · 버전 bump · changelog
  - @1.0 superseded 마킹 · 포크자 알림
- 홈페이지 족보 · 워크샵 메모 자동 업데이트

④ 신입 온보딩

기존 선배마다 다른 답, 한 달 헤맴

새 방식 `share.damilab` 에서 「논문 정리」 태그로 필터

- 추천(shared) · 변형(personal) · 원조(yunji) 한 화면
- why · how-to-use · 족보 · 코멘트 확인
- **한 시간 안에 첫 논문 정리**

효과: 선배한테 물어보는 게 **병목이 안 됨**.

예시 5·6·7. 큐레이션 · 라이팅 · 자연 발생

■ 세 시나리오 — 기여가 자연스럽게 일어나는 경로

⑤ 큐레이션 교수님이 Anthropic 프롬프트 캐싱 가이드를 홈페이지 큐레이션 섹션에 저장 → git commit 자동 → 민석 멘션 슬랙 알림 → **영구 보존 · 검색 가능**. 슬랙 휘발 문제 해결.

⑥ 라이팅 (윤지 ICML) `minseok/paper-writing-assist@1.1` + `robot-team/latex-format-check@0.3` 을 `search` 가 찾아주고, `fork` 가 두 부모 기반 새 스킬 `yunji/icml-writing-pipeline` 생성 → 족보에 **다중 부모**로 표시. 하이브리드 스킬도 자연스럽게 표현됨.

⑦ 자연 발생 (승현 FAVOR) FAVOR TDI 리비전 중 reviewer 코멘트 분류가 반복됨 → Claude 가 작업 로그 기반 스킬 초안 생성 → 본인 작업에 즉시 활용 → 워크샵 공유 → 민석 포크 → 몇 주 후 승격. **「스킬 만들자」 부담 없이** 연구 작업에서 자연 발생.

공통 패턴: 특별한 의지 없이도 **연구·업무 흐름 속에서** 기여가 일어나도록 시스템이 설계되어 있음. 강요 없이 생태계가 돌아가는 이유.

5. 단계별 로드맵

Phase 0 ~ 4 타임라인 개관

■ 원칙

- **수동 실험 → 기초 인프라 → 엔진 → 승격 → 확장** 의 5단계
- 처음부터 풀 자동화 금지, **먼저 수동으로 해보고** 반복되는 것만 자동화

Phase 0 · 수동	**워크플로우 살아보기** — frontmatter 규칙 초안, 전원 스킬 1개 수기 마이그레이션, 귀찮음 로그로 자동화 대상 발굴
Phase 1 · 인프라	**구조를 코드로 박기** — `damilab-skills` repo · validator · GitHub Actions · 읽기 전용 홈페이지 MVP · 본인 스킬 마이그레이션
Phase 2 · 엔진	**생태계 엔진 가동** — Sveltia CMS 연동 · 족보 트리 · 첫 메타 스킬 `skill-fork` · 실제 포크 경험 축적
Phase 3 · 승격	**생태계 완성** — `skill-promote` · Slack 봇 알림 · diff 뷰 · 정기 목요일 워크샵 체계화
Phase 4 · 확장	**전원 합류 · 성숙** — 브라질팀 합류 · 슬랙 자료 이전 · 코멘트/토론 고도화 · 외부 공개 범위 검토

Phase 0·1 상세 — 수동 실험 + 기초 인프라

■ 집중 포인트

- Phase 0 은 **도구 없이** 살아본다. 여기서 발굴된 반복 작업이 Phase 1·2 자동화 대상
- Phase 1 은 **읽기 전용 홈페이지** 까지. 글쓰기는 아직 없음 (CMS 는 Phase 2)

🟢 Phase 0. 수동 실험기

- **교수님 + 민석**
  - frontmatter 규칙 초안 작성 → 전원 공유
- **전원**
  - 본인 스킬 1개 수기 metadata 작성 · repo 업로드
  - 첫 워크샵에서 「이렇게 만들어봤다」 공유
  - **귀찮음 로그** — Phase 1 자동화 대상 발굴

산출물: frontmatter 규칙서 · 연구원별 첫 스킬 · 귀찮음 로그

🔵 Phase 1. 기초 인프라

- **교수님**: 방향 · 우선순위 최종 승인
- **민석**: `damilab-skills` repo · frontmatter validator · GitHub Actions
- **이윤석**: `share.damilab` MVP — 정적 카탈로그, **읽기 전용**
- **전원**: 본인 스킬들 새 구조로 마이그레이션

산출물: 읽기 전용 홈페이지 · 구조화된 스킬 repo

Phase 2·3 상세 — 엔진 가동 + 승격 시스템

■ 집중 포인트

- Phase 2 에서 비로소 **글쓰기 가능**한 홈페이지와 첫 메타 스킬 등장
- Phase 3 에서 **승격 + 알림** 이 완성되어 생태계가 자기 유지 상태

Phase 2. 엔진 돌리기

- **이윤석**: Sveltia CMS 붙이기 · 족보 트리 시각화
- **민석**: 첫 메타 스킬 `damilab-skill-fork`
- **정승현**: FAVOR 리비전 → **논문 작성 스킬** 정리 · 공유
- **윤지**: ICML 워크숍 제출 → **논문 라이팅 스킬** 실험
- **전원**: 실제 포크 · 수정 · 피드백

산출물: 글쓰기 홈페이지 · 첫 메타 스킬 · 족보 · 작동 예시 3~5개

Phase 3. 승격 + 알림

- **민석**: `damilab-skill-promote` · 승격 워크플로우 확정
- **교수님 + 민석**: Slack 봇 (포크 · 승격 · 업데이트 이벤트)
- **이윤석**: 족보 트리 고도화 · diff 뷰 개선
- **로봇팀**: 외부 교수 협업 맥락의 **PPT 제작 스킬** 기여
- **전원**: 주간 목요일 워크샵 체계화

산출물: 완성된 생태계 · 정기 워크샵 루틴 · 알림 시스템

Phase 4 + 진행형 과제

■ 남은 것들

- Phase 4 는 시스템이 **성숙한 후**의 확장 단계. 브라질팀 합류와 외부 공개 검토
- 「진행형」 과제는 특정 Phase 에 묶이지 않고 **계속 조율**

🔴 **Phase 4. 확장 및 합류**

- **브라질팀 (2명)**: 학회 종료 후 합류, 본인 영역 스킬 기여
- **스킬 큐레이션 페이지**: 슬랙 자료를 홈페이지로 이전
- **코멘트 / 토론** 시스템 고도화
- **외부 공개 범위** 검토

진행형 과제

- **연구실 공통 폴더 구조/원칙** 지속 조율 — 오늘 결론 내는 항목 아님
- frontmatter 규칙 · 승격 기준 · 개인/공용 경계 **계속 진화**

원칙: 완벽한 규칙을 먼저 만들고 시작하지 않음. **쓰면서 규칙이 진화**하도록 구조를 유연하게.

6. 역할과 의미

팀 / 역할 분담

■ 배치 원칙

- **프로젝트 리드는 김민석 단일** — 의사결정 흐름과 책임이 명확해야 속도
- 이윤석은 기술 co-contributor (`research.dami lab` 은 이윤석의 **별도 트랙**, 이 로드맵 밖)
- 나머지 연구원은 **본인 연구 속에서 자연스럽게 기여** — 별도 부담 없음

역할	담당	기여 포인트
프로젝트 리드	김민석	총괄 · 메타 스킬 · repo 구조 · 승격 워크플로우 · Slack 봇
기술 서포트	이윤석	`share.dami lab` 프론트 · CMS 연동 · 족보 트리
방향 · 원칙	교수님	frontmatter 규칙 · 승격 기준 · 우선순위
연구 중심 기여	정승현	FAVOR 리비전 속 **논문 작성 스킬** 자연 기여
연구 중심 기여	김윤지	ICML 제출 경험 기반 **논문 라이팅 스킬**
특수 영역	로봇팀 (2명)	외부 협업 특성의 **PPT · 문서 스킬**
후속 합류	브라질팀 (2명)	학회 종료 후 Phase 4 참여
공통	전원	매주 목요일 워크샵 · Phase 0 실험 · 포크 · 수정

설문 Pain Point → 해결 매핑

■ 효과

- 설문에서 나온 다섯 문제를 **이 시스템이 동시에** 해결
- 단순 도구가 아니라 **연구실의 차별점**으로 작동

Pain Point	해결 방식
스킬 중복	족보에서 이미 있는지 바로 파악
뭘 어떻게 쓸지	why + how-to-use + 예시 **필수**
슬랙에 묻힘	카탈로그 + 큐레이션 페이지
하네스 세팅	메타 스킬이 규칙 자동 적용
부담 편중	전원이 자연스럽게 기여

다미랩의 차별점

- 다른 연구실: 각자 만들어 자기만 씀
- **다미랩**: 스킬 진화 과정이 **자동 기록 · 공유**되는 생태계
- **1년 후**: 대량의 족보 · 맥락 · 토론 데이터 = 연구실 자산

Tool-as-Product: 우리가 쓰는 도구가 연구실의 자산. 신입도 시스템만 보면 「우리 연구실은 이렇게 일한다」 학습.

세미나 핵심 메시지 3가지

■ 세 문장으로 요약

- **무엇을** 만들려고 하는가, **어떻게** 만들 것인가, **누가** 참여하는가
- 기술 인프라만으로는 부족, **사회적 프로세스** 가 동등하게 중요

① **무엇을**

스킬이 자연스럽게 포크·진화·공유되는 **생태계**.
족보는 자동으로 남고, 메타 스킬이 반복 작업을 처리

→

② **어떻게**

Git-backed CMS + 메타 스킬 = 기술 인프라.
매주 목요일 워크샵 = 사회적 프로세스.
단계적 Phase 로 점진 구축

→

③ **모두의 역할**

주도 3인만 있는 게 아님.
본인 연구 속 스킬을 **자연스럽게 기여**,
남이 만든 것을 **포크**,
워크샵에서 **공유**

강조: 「참여 강제」 가 아니라 **「참여가 자연스럽게 일어나는 구조 설계」**. 포크는 자유, 승격은 합의.

주의사항 · 함정

■ 피해야 할 것들

- 이전 프로젝트들에서 반복된 실패 패턴을 정리. **구축 과정에서 체크리스트로** 활용

함정 1 **처음부터 풀 자동화 시도 금지** — 먼저 수동으로 해보고, 뭐가 반복되는지 확인한 뒤 **그것만** 자동화

함정 2 **규칙 완벽주의 금지** — frontmatter 규칙은 초안부터 시작. 쓰면서 진화

함정 3 **전원 동시 참여 강요 금지** — 주도 3인이 MVP 만들면, 나머지는 준비되는 대로 합류

함정 4 **기술로만 해결 X** — 매주 목요일 워크샵 같은 **사회적 인프라**가 동등하게 중요

함정 5 **완성 후 공개 금지** — MVP 1 부터 전원이 쓰기 시작. 쓰면서 피드백으로 MVP 2, 3 제작

공통 원칙: **작게 시작 · 빠르게 피드백 · 점진 진화**. 이 로드맵의 모든 단계에 녹아 있는 태도.

Thank you